Expert Systems with Applications xxx (2015) xxx–xxx

Contents lists available at ScienceDirect

Expert Systems with Applications

journal homepage: www.elsevier.com/locate/eswa

The continuous Berth Allocation Problem in a container terminal with multiple quays

Pablo Frojan, Juan Francisco Correcher, Ramon Alvarez-Valdes *, Gerasimos Koulouris, Jose Manuel Tamarit

University of Valencia, Department of Statistics and Operations Research, Doctor Moliner 50, 46100 Burjassot, Valencia, Spain

ARTICLE INFO

Article history:
Available online xxxx

Keywords:
Berth Allocation Problem
Container terminals
Priority rules
Metaheuristics
Genetic algorithm

ABSTRACT

This paper extends the study of the continuous Berth Allocation Problem to the case of multiple quays, which is found in many container terminals around the world. Considering multiple quays adds a problem of assigning vessels to quays to the problem of determining berthing times and positions for each incoming vessel.

This problem has not been considered in the literature at the operational level addressed in this study. In order to solve it, we have developed first an integer linear model and then a genetic algorithm which works on sequences of vessels that are decoded by a constructive algorithm. A local search procedure helps to improve further the solutions produced by the genetic algorithm.

We have conducted an extensive computational experiment, adapting existing instances to the case of multiple quays. We have also adapted our genetic algorithm to the case of a single quay and tested it on previously reported instances, showing its superior performance compared with existing approaches. Finally, we have developed a random instance generator for the problem with multiple quays, enabling us to conduct a systematic study of the factors affecting the complexity of the problem.

© 2015 Elsevier Ltd. All rights reserved.

1. Introduction

Container terminals appeared in the late 1960's as a consequence of the traffic of specialized container ships between Europe and the USA. The use of containers has become widespread due to their important advantages: less product packing, less deterioration, and much greater productivity in the different handling phases. Moreover, it makes the intermodal transportation of goods much easier, because the transfer from vessels to rail or road is simplified. Consequently, the number and size of container terminals have continuously increased. In 2013, the total volume of container loading/unloading operations surpassed 650 million *TEUs* (*Twenty-foot Equivalent Unit*) (UNCTAD, 2014). Ports like Shanghai and Singapore moved more than 30 million TEUs. In Europe, Rotterdam moved almost 12 million, followed by Hamburg with more than 9 million. In Spain, the largest volume of container traffic was that of Algeciras, with 4.5 million, and Valencia, with 4.3 million TEUs. Overall, the percentage increase in the decade 2001–2010 was 206%.

The productivity of these busy container terminals is highly dependent on the terminal handling system, as well as on the efficient berth allocation of calling vessels. In this paper we study the *Berth Allocation Problem (BAP)*, which consists of allocating to each vessel a berthing position and a time to perform its loading/unloading operations. Depending on the spatial structure of the terminal, several types of problems can be distinguished. Basically, we can differentiate between discrete and continuous cases. In the *discrete* case, the quay is divided into fixed sections and each section can be occupied by at most one vessel, irrespective of its length. In the *continuous* case, the quay is not divided and a vessel can berth at any position at the quay, depending only on the position of other vessels. Considering a continuous quay adds flexibility but also increases the complexity of the problem.

Up to now, in all previous studies dealing with the continuous BAP, the quay has been considered a single continuous line of a given length. However, when we started to study this problem, we discovered that one of the terminals in the port of Valencia (Spain) consists of two quays, and considering it as a single line did not seem a good approximation. Looking at terminal layouts all over the world, we found many other terminals with several

* Corresponding author. Tel.: +34 963544308; fax: +34 963543238.
E-mail addresses: xneo20@gmail.com (P. Frojan), juan.correcher@uv.es (J.F. Correcher), ramon.alvarez@uv.es (R. Alvarez-Valdes), gerasimoskoulouris@hotmail.com (G. Koulouris), jose.tamarit@uv.es (J.M. Tamarit).

http://dx.doi.org/10.1016/j.eswa.2015.05.018
0957-4174/© 2015 Elsevier Ltd. All rights reserved.

quays. In Singapore, Keppel terminal is divided into four quays and Tanjong Pagar and Brani terminals into three. Maher and APM terminals in the Port of New York and New Jersey (USA) have two quays each, while Terminal 1 of Jebel Ali Port (Dubai) has three quays. Many other examples can be observed in other ports. It could be argued that algorithms designed for single quay terminals can be adapted to the multiple quay case. Nevertheless, addressing the problem as it really is, with multiple quays, can lead to better algorithms and, more importantly, will allow us to include more realistic characteristics. Conditions such as the fact that a given vessel cannot moor at a given quay for technical or contractual reasons, or the assignment of different costs for mooring at different quays, are easily added to algorithms specifically designed for terminals with multiple quays.

This paper extends the dynamic and continuous Berth Allocation Problem to the case of multiple quays. Considering multiple quays to moor a set of calling vessels adds a problem of assigning vessels to quays to the basic problem of deciding the time and position for each vessel at the quay. While the basic problem can be represented in a single space–time diagram, the problem with multiple quays requires a multiple space–time representation, as can be seen in Fig. 1, in which each space–time diagram shows the distribution of the vessels assigned to each quay. The horizontal axis shows the time, in hours, and the vertical axis the length, in meters. The first quay has a length of 800 m and 5 vessels are berthed. Quay 2 has 600 m and 3 vessels, while quay 3 is 1000 m long and 4 vessels have been assigned to it. Each vessel is represented by a rectangle, whose length is the handling time and whose height represents the vessel length.

In order to solve this problem we have worked in two complementary directions. We first developed an integer linear model with the objective of minimizing the sum of all operational costs involved. That includes the usual waiting and delay costs for each vessel and also the cost of assigning vessels to quays and, for each vessel, the cost of deviation from its desired berthing position at the quay. In a second phase we developed a genetic algorithm in order to solve problems of realistic size in short computing times. The BAP allows a natural representation using sequences of vessels that can be used by the genetic operators to evolve them towards high quality solutions. Vessel sequences are decoded by using a fast constructive algorithm. The solutions obtained by the genetic algorithm are further improved by a local search procedure.

The integer linear model and the genetic algorithm have been tested on a set of instances generated by adding more quays to the one-quay instances of Park and Kim (2002). Nevertheless, in order to assess the performance of the proposed algorithms, we have also solved the one-quay instances of Lee, Chen, and Cao (2010) and Cordeau, Laporte, Legato, and Moccia (2005), adapting our algorithm to one quay and using the objective function proposed in each study. The results show that our genetic algorithm outperforms both the GRASP algorithm by Lee et al. (2010) and the Tabu Search algorithm by Cordeau et al. (2005). We have also developed a random instance generator in which all the parameters (number and length of quays and vessels, arrival and departure times, relative importance of costs) can be controlled, thus allowing us to study the effect of each element on the complexity of the problem.

The rest of the paper is organized as follows. After the literature review in Section 2, in Section 3 we describe the characteristics of the problem and then in Section 4 we present the mathematical formulation for the continuous dynamic *BAP* with multiple quays. In Section 5 we present our metaheuristic solution approach, describing first the elements of the genetic algorithm, then the constructive algorithm for decoding the sequences, and finally the local search procedure. Section 6 presents the numerical experiments with the integer formulation and the genetic algorithm. Finally, in Section 7 we draw some conclusions and discuss future work.

2. Related work

Berth allocation is considered one of the key issues in a container terminal. Therefore, it is not surprising that the many variants of the problem have been the object of extensive research in recent years. In their first survey (Bierwirth & Meisel (2010)) reviewed 53 papers published from 1994 to 2008, while in their second survey (Bierwirth & Meisel, 2015) they review 79 new papers that have appeared since 2009. These surveys cover all types of Berth Allocation Problems, classified by the spatial layout (discrete, continuous, or hybrid), the temporal attributes of the vessels (static, dynamic, with due dates), the types of handling times (fixed, depending of the position at the quay, depending on the number of quay cranes assigned, depending on the quay crane schedules) and the measures of performance, including different

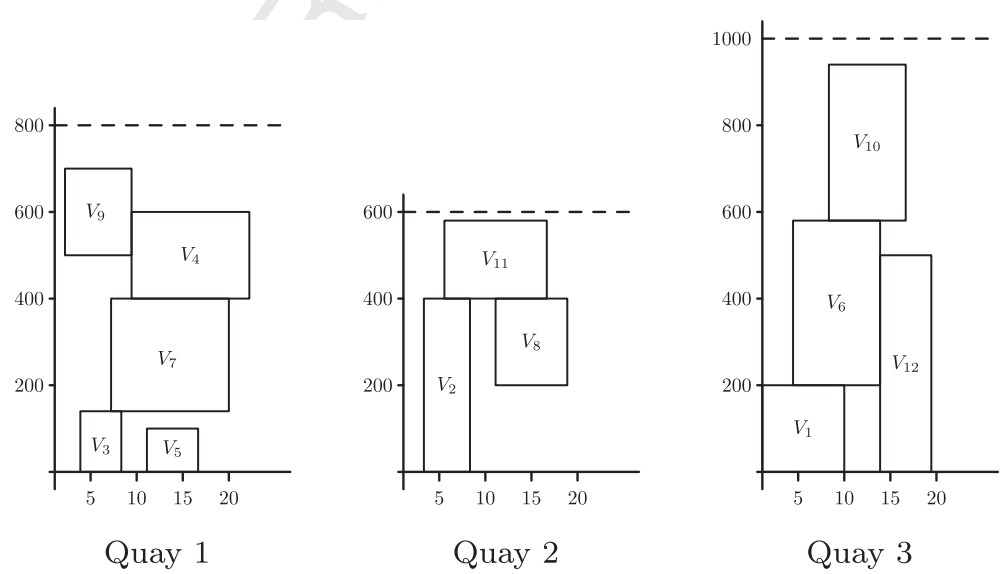


Fig. 1. A BAP solution with 3 quays and 12 vessels.

Please cite this article in press as: Frojan, P., et al. The continuous Berth Allocation Problem in a container terminal with multiple quays. *Expert Systems with Applications* (2015), http://dx.doi.org/10.1016/j.eswa.2015.05.018

costs related to the waiting times, the handling times, the completion times, the deviation of the vessel from the desired position at the quay, or other factors such as fuel consumption.

In this study we focus on the dynamic and continuous Berth Allocation Problem with fixed vessel handling times (for recent studies on the discrete case, see, for instance, Ting, Wu, & Chou (2014) and Oliveira, Mauri, & Lorena (2012)). The continuous *BAP* was introduced by Li, Cai, and Lee (1998) as a machine sequencing problem. Guan, Xiao, Cheung, and Li (2002) also maintained the parallelism between task scheduling and berth allocation, proposed a heuristic priority rule, and performed a worst-case analysis.

Guan and Cheung (2004) move away from the machine scheduling scheme and propose a tree-search procedure for minimizing the total weighted length of stay of vessels in the port. Wang and Lim (2007) consider the possibility of not berthing all the vessels and minimize an objective function that combines rejection costs, delay costs, and the cost of berthing a vessel away from its desired position. They propose a *Stochastic Beam Search (SBS)*. Their computational study shows that *SBS* is able to solve instances with up to 400 vessels and that it outperforms the *Simulated Annealing* algorithm by Dai, Lin, Moorthy, and Teo (2008). Park and Kim (2002) develop an integer linear model and solve it using Lagrangian Relaxation for small instances. Kim and Moon (2003) solve the problem described by Park and Kim using *Simulated Annealing*. Cordeau et al. (2005) model the discrete *BAP* as a vehicle routing problem with time-windows and solve it by a *Tabu Search* algorithm which they then adapt to the continuous case. More recently, Lee et al. (2010) have developed a new integer linear model and two *GRASP* algorithms which are able to solve instances with up to 200 vessels, showing that they outperform the *SBS* by Wang and Lim (2007).

Lim (1998) considers a version of the problem in which the berthing times are given and only the berthing positions have to be decided, and the objective is to minimize the total length of the quay required to serve all the vessels. He represents the problem as a graph and transforms it into one of assigning directions to the edges to minimize the longest path. Tong, Lau, and Lim (1999) apply an *Ant Colony Algorithm (ACO)* to Lim's representation.

Although most of the studies consider a single objective function, usually combining different costs into a linear expression, there is an increasing number of papers that consider a multi-objective approach. Zhen and Chang (2012) include the maximization of the schedule robustness, which is measured taking into account the time between vessels occupying the same berth space. Xu, Chen, and Quan (2012) also consider uncertainty in the vessel arrival and handling times and include in their model a buffer time between vessels, with the objective of optimizing a combination of service quality and solution robustness. Du, Chen, Quan, Long, and Fung (2011) introduce fuel consumption and vessel emissions into the problem. They end up with a bi-objective non-linear model, but by using SOCP (Second Order Cone Program) and the ε-constraint approach for bi-objective optimization, they show that, if the speed of vessels is controlled, it is possible to reduce fuel consumption and vessel emissions significantly, while keeping waiting costs of the vessels at the terminal at a satisfactory level.

The continuous BAP with multiple quays has not been studied at the operational level that we consider in this study, but there are several recent papers dealing with multi-terminal allocation problems at a tactical or strategic level. Hendriks, Laumanns, Lefeber, and Udding (2010) consider the case in which several terminals are managed by the same operator and the problem cannot be solved considering one terminal at a time. Their objective is the strategic allocation of cyclically calling vessels, taking into account operational costs at each terminal and transportation costs between terminals. The solution approach is based on an integer linear model and is able to solve real-life instances in satisfactory computing times. Lee, Jin, and Chen (2012) also consider several terminals in their study of a terminal and yard allocation problem for a transshipment hub. A two-level heuristic algorithm produces high quality solutions in an efficient way.

Also related to the continuous BAP with multiple quays is the case of hybrid layouts, in which the quay is first divided into sections, but the condition of assigning exactly one vessel to each section is relaxed, allowing more than one small vessel to occupy the same section or a large vessel to occupy more than one section. An example is the recent work by Imai, Nishimura, and Papadimitriou (2013), in which the sections into which the quay is divided are designed for mega-ships and can be occupied by two smaller feeders. Even more closely related to the work developed here is the study by Cheong, Tan, Liu, and Lin (2010), who also consider the quay divided into sections, but with each section able to accommodate many vessels, as long as they do not overlap and the total length of the vessels being served simultaneously does not exceed the length of the section. In this respect, it is the same layout we consider here, although they do not consider other elements, such as the assignment costs of vessels to sections or the desired position of each vessel in each section, that allow us to exploit the characteristics of the multiple quay layout. Instead, they focus on a multi-objective approach, considering the makespan, the total waiting time of the vessels, and the degree of adherence to a predetermined priority schedule, and study their joint behavior in an extensive simulation study.

3. Problem description

In the continuous BAP with multiple quays we have a set of quays with known lengths and a set of vessels with known characteristics (length, arrival time, handling time, desired departure time, desired position at each quay, ...), and the problem consists of deciding the berthing time, the quay, and the position at the quay for each vessel, in such a way that the total assignment cost is minimized. The main elements in the problem are:

- A set Q of container terminal quays. Each quay $q \in Q$ has a length L_q.
- A set V of vessels to berth.
  For each vessel $i \in V$, the following characteristics are known:
  - Length: l_i.
  - Estimated handling time (loading/unloading): b_i.
  - Estimated arrival time: a_i.
  - Desired departure time: s_i.
  - Ideal position at quay q: d_{iq}.
  - Cost per unit time of waiting for berth: C_i^w.
  - Cost per unit time of delay with respect to the desired departure time: C_i^d.
  - Assignment cost of vessel i to quay q: C_{iq}^a.
  - Cost of unit deviation of vessel i from its ideal position at quay q: C_{iq}^p.

The cost has four components: the waiting cost, where waiting time is defined as the difference between the estimated arrival time and the assigned berthing time; the delay cost, where delay is defined as the difference between the desired departure time and the end of the handling of the vessel; the cost of assigning the vessel to the quay, and the cost of the deviation of the berthing position from the ideal position at the quay. Some assignments of

UNCORRECTED PROOF

Please cite this article in press as: Frojan, P., et al. The continuous Berth Allocation Problem in a container terminal with multiple quays. *Expert Systems with Applications* (2015), http://dx.doi.org/10.1016/j.eswa.2015.05.018

vessels to quays can be forbidden for contractual reasons or because the draft or the machinery in the quay are not suitable for the vessel, but some other possible assignments can simply be made less desirable by assigning high assignment costs C^a_{iq}. Once a vessel i is assigned to a quay q, not all the positions along the quay are equally desirable. Cost C^p_{iq} is the cost of deviation from the desired position at quay q. In other planning problems, deviation from the ideal position, which reflects the location of the containers to be loaded or the slots in which to place the containers to be unloaded, may affect the handling time, but in this *BAP* the handling time is considered fixed and the deviation is included as a cost, because if the containers have to be moved a longer distance and the handling time is fixed, more resources have to be allocated.

The problem is based on the following assumptions:

- Once a vessel is moored, the berthing position is fixed. Once started, the process cannot be interrupted, to avoid additional costs.
- Handling times for the vessels are considered fixed and independent of the berthing position. This assumption is reasonable if the quay has enough machinery and workers for the loading/unloading tasks at any moment (Lim, 1998).
- Each vessel has a different relative importance. Therefore, the four types of costs are specific to each vessel.
- The inter-ship clearance is included in the vessel length. In general, for vessels longer than 130 m, this clearance corresponds to 10% of the vessel's length. For small vessels, the minimum clearance is 10 m. (Lee et al., 2010).
- The time for docking and undocking maneuvers is considered to be included in the vessel handling time.
- Each position at the quay can accommodate one vessel at a time.
- Specific conditions preventing a given vessel from mooring at a given quay, such as contract conditions, quay depth, vessel draft, or specific machinery requirements, are not explicitly taken into account, but can be included by considering very high assignment costs C^q_i for vessel i at quay q.

4. A mixed integer linear model for continuous *BAP* with multiple quays

We define variables:

- t_i = berthing time of vessel i
- m_{iq} = 1, if vessel i is moored at quay q; 0, otherwise
- p_i = berthing position of vessel i at the quay to which it is assigned
- σ_{ij} = 1, if in the space–time diagram vessel i is completely to the left of vessel j, that is, vessel i is completely processed before vessel j; 0, otherwise
- δ_{ij} = 1, if in the space–time diagram vessel i is completely below vessel j, that is, vessel i is positioned completely to the right of vessel j (looking from the quay); 0, otherwise
- r_i = delay of vessel i
- e_i = deviation of the assigned position of vessel i from its ideal position at the quay to which it is assigned.

Using these variables and the elements defined in the previous section, a model for the continuous *BAP*, which extends the Park and Kim (2002) model, is the following:

$$Min \sum_{i\in V}(C^w_i(t_i - a_i) + C^d_i r_i) + \sum_{q\in Q}\sum_{i\in V}(C^a_{iq} m_{iq} + C^p_{iq} e_i) \quad (1)$$

s.t.

$$t_j - (t_i + b_i) - (\sigma_{ij} - 1)T \geqslant 0 \quad i,j \in V, i \neq j \quad (2)$$
$$p_j - (p_i + l_i) - (\delta_{ij} - 1)L_{max} \geqslant 0 \quad i,\ j \in V, \quad i \neq j \quad (3)$$
$$\sigma_{ij} + \sigma_{ji} + \delta_{ij} + \delta_{ji} \geqslant m_{iq} + m_{jq} - 1 \quad i,\ j \in V, i \neq j,\ q \in Q \quad (4)$$
$$p_i + l_i \leqslant \sum_{q\in Q} m_{iq} L_q \quad i \in V \quad (5)$$
$$\sum_{q\in Q} m_{iq} = 1 \quad i \in V \quad (6)$$
$$t_i \geqslant a_i \quad i \in V \quad (7)$$
$$r_i \geqslant t_i + b_i - s_i \quad i \in V \quad (8)$$
$$e_i \geqslant p_i - \sum_{q\in Q} d_{iq} m_{iq} \quad i \in V \quad (9)$$
$$e_i \geqslant \sum_{q\in Q} d_{iq} m_{iq} - p_i \quad i \in V \quad (10)$$
$$\sigma_{ij}, \quad \delta_{ij} \in \{0,1\} \quad i,j \in V, \quad i \neq j, q \in Q \quad (11)$$
$$m_{iq} \in \{0,1\} \quad i \in V, \quad q \in Q \quad (12)$$
$$t_i,\ p_i,\ r_i,\ e_i \geqslant 0 \quad i \in V \quad (13)$$

where $L_{max} = max_{q\in Q}\{L_q\}$ and T is un upper bound on the total time required to service all the vessels.

The objective function (1) minimizes the sum of the four types of costs. The first term corresponds to the waiting and delay costs for each vessel and the second to the assignment and deviation costs for each vessel and quay. Constraints (2)–(4) prevent overlapping in time and space of vessels assigned to the same quay. Constraints (5) ensure that the position at the quay is valid and constraints (6) that every vessel is assigned to a quay. Constraints (7) do not allow vessels to be moored before their arrival. Constraints (8) define the delay for each vessel, which cannot be negative. Constraints (9) and (10) define the deviation of a vessel from its desired position at the quay to which it has been assigned. Constraints (11)–(13), define the variable types. In constraints (7) the start of the handling of each vessel is equal to or greater than its arrival time. In other planning models it is possible to bring forward the arrival of a vessel, with an associated cost, and then t_i can be made lower than the initially calculated a_i. This could easily be accommodated in the model, but as BAP models usually include constraints (7) we have retained them.

5. Solution approach

The integer linear program presented in the previous section is very useful for describing the elements of the problem and their interactions, but it can produce optimal solutions only for small-medium size instances, as will be detailed in Section 6. Therefore, in order to solve large instances efficiently, as those solved by Lee et al. (2010), with up to 200 vessels, a metaheuristic algorithm is necessary. In this study we have decided to develop a genetic algorithm, because it has produced very good results in packing problems (Gonçalves & Resende, 2013; Hopper & Turton, 2001; Iori, Martello, & Monaci, 2003), which are very closely related to the BAP, and have also been successfully applied to other versions of the BAP (Nishimura, Imai, & Papadimitriou, 2001). In this section we describe first the elements of our genetic algorithm, in which the representation of a solution is a sequence of vessels, then the decoding algorithms we have developed to produce a feasible solution for the BAP from a vessel sequence, and finally the local search procedure.

5.1. A genetic algorithm

The basic decision when designing a genetic algorithm is the way in which a solution is represented. Certain genetic operators

UNCORRECTED PROOF

Please cite this article in press as: Frojan, P., et al. The continuous Berth Allocation Problem in a container terminal with multiple quays. *Expert Systems with Applications* (2015), http://dx.doi.org/10.1016/j.eswa.2015.05.018

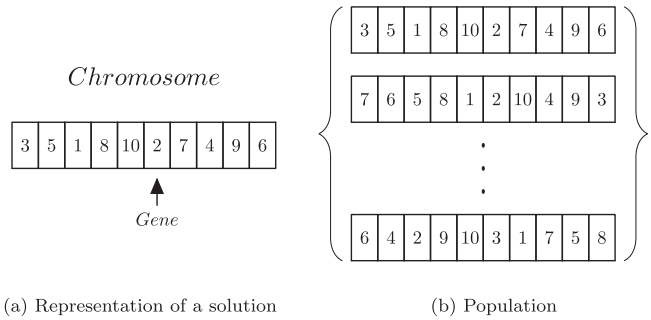


Fig. 2. Elements of the genetic algorithm.

Table 1
Priority rules used for building the initial population.

Estimated arrival time	Non-decreasing a_i
Handling time	Non-increasing b_i
Maximum length	Non-increasing l_i
Minimum length	Non-decreasing l_i
Waiting cost	Non-increasing C_i^w
Delay cost	Non-increasing C_i^d
Departure time	Non-decreasing s_i
Area	Non-increasing $AB_i = l_i b_i$
Weighted area 1	Non-increasing $APD_i = AB_i/C_i^d$
Weighted area 2	Non-increasing $APM_i = AB_i C_i^d$
Weighted handling time	Non-increasing $TPP_i = C_i^d b_i$
Slack	Non-decreasing $h_i = s_i - (a_i + b_i)$
Weighted slack	Non-decreasing $h_i^b = h_i/C_i^d$
Area-slack ratio	Non-increasing $APH_i = APM_i/h_i$
C1	Non-decreasing $(a_i + b_i)/s_i$
C2	Non-decreasing $(a_i - b_i)/s_i$
C3	Non-decreasing $(a_i + b_i + l_i)/(a_i + s_i)$
C4	Non-decreasing $(b_i + l_i)/(a_i + s_i)$
C5	Non-increasing $b_i s_i C_i^d$
C6	Non-increasing $h_i^b/C2_i$
C7	Non-decreasing $h_i^b/C2_i$
C8	Non-increasing $C_i^w(b_i + l_i)/(a_i + s_i)$
C9	Non-increasing $C_i^d(b_i + l_i)/(a_i + s_i)$
C10	Non-increasing $((a_i - b_i - s_i)/(C_i^d)) * (b_i + l_i)/(a_i + s_i)$
C11	Non-increasing $(a_i b_i)/s_i$
Random order	

(crossover and mutation) are defined according to this representation, and applied to the individuals in the population in order to make it evolve and produce high quality solutions. The main elements of our genetic algorithm are:

- **Representation**
  Instead of using the layout representation of a solution, as shown in Fig. 1, we use a simpler representation in which the chromosome is an ordered list of vessels, as appears in Fig. 2(a). Therefore, its length is the number of vessels. In order to obtain a solution, the chromosome is decoded by using the procedure described in the next subsection.
- **Initial population**
  The initial population is composed of 30 individuals, 26 corresponding to the sequences obtained by using the 26 priority rules described in Tables 1 and 4 that are randomly generated, different from those provided by the priority rules. The first group of rules corresponds to individual characteristics of the vessels, while a second group combines these features into more complex rules. Previous studies have shown that including good solutions in the initial population produces good results (see, for instance, Vallada & Ruiz (2011)). In this paper we go one step further and form most of the initial population using priority rules. A possible disadvantage could have been that this population was too homogeneous, because that could have provoked a premature convergence. However, the priority rules order the vessels in different ways and produce quite different solutions, and the four randomly generated solutions add more diversity to the initial population. In fact, a preliminary analysis revealed that by using this initial population we obtained better solutions than with a completely random initialization.
- **Measure of fitness**
  As a measure of fitness we take the total assignment cost of all the vessels. As we are dealing with a minimization problem, the smaller the objective function value, the higher the fitness value.
- **Reproduction**
  We choose 15 pairs, using the roulette method in which each individual has a probability proportional to its fitness. The operator used is the 1-point crossover in which the parent sequences are cut at a randomly selected position and each child gets the first subsequence of one of the parents and completes it with the sequence of the other. Thus, each child has a complete ordered sequence of vessels which corresponds to a feasible solution to the problem (see Fig. 3(a)).
- **Mutation**
  Each child obtained by the crossover operation has a probability P_m of suffering a mutation, which consists of exchanging the positions of two randomly selected vessels (see Fig. 3(b)).
- **New population**
  A new individual obtained through the reproduction and mutation processes forms part of the next generation if its fitness is better than the fitness of the worst individual in the current generation.

5.2. Algorithms to decode a vessel sequence into a BAP solution

We have developed two constructive algorithms that build a feasible solution from an ordered list of vessels. Both algorithms take one vessel at a time and look for its least-cost assignment at each quay, each available position and each available berthing time at which it does not overlap the vessels previously assigned.

5.2.1. Exploratory Constructive Algorithm

The first algorithm, denoted as the *Exploratory Constructive Algorithm (ECA)*, looks for the best assignment for each vessel by

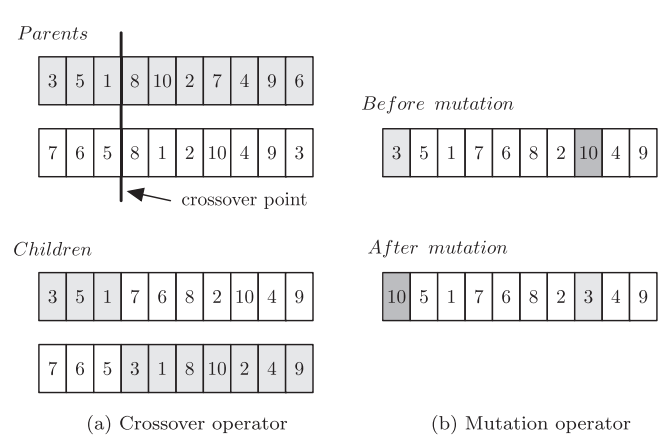
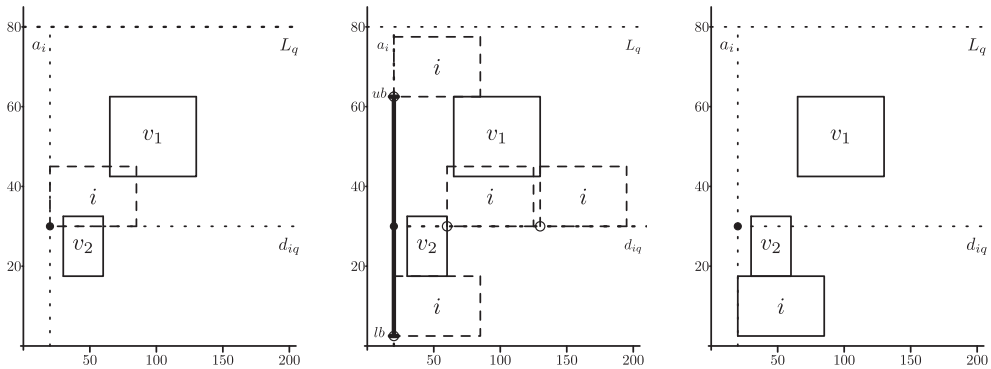


Fig. 3. Genetic operators.

UNCORRECTED PROOF

Please cite this article in press as: Frojan, P., et al. The continuous Berth Allocation Problem in a container terminal with multiple quays. *Expert Systems with Applications* (2015), http://dx.doi.org/10.1016/j.eswa.2015.05.018

building and exploring a set of candidate assignments, ordered by non-decreasing cost. For each vessel, this set initially contains the assignment in which the berthing time is its arrival time and its berthing position is its desired position at the quay with the lowest assignment cost. From the set of candidates we take the least-cost assignment and check whether berthing the vessel at this time, quay, and position overlaps any other previously assigned vessel. If it does, new candidate assignments are identified around this vessel such that this overlap is avoided. In order to avoid redundant tests, we also identify the interval of positions that will be forbidden because any position inside the interval would lead to a new overlap. The process is repeated, taking the least-cost

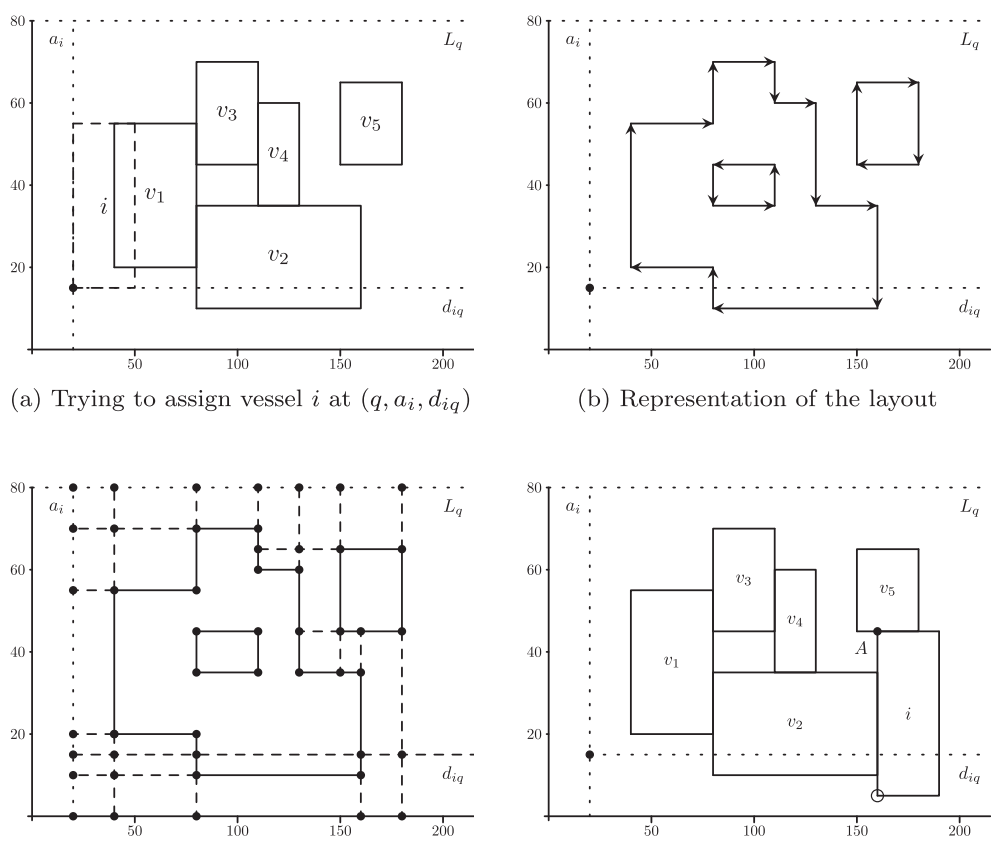


(a) Trying to assign vessel i to its best position

(b) Get new candidates surrounding v_1 and v_2 and identify the forbidden interval (lb, ub) at $t = a_i$

(c) Vessel i assigned to the best feasible position

Fig. 4. Process of the Exploratory Constructive Algorithm.



(a) Trying to assign vessel i at (q, a_i, d_{iq})

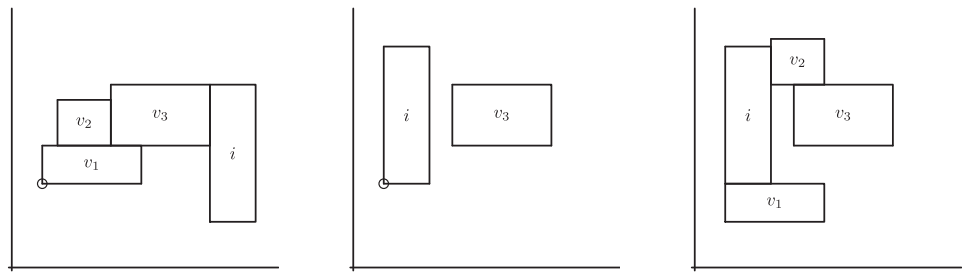
(b) Representation of the layout

(c) Looking for relevant points

(d) Assignment of vessel i to a position obtained from point A

Fig. 5. Process of the Analytic Constructive Algorithm.

Please cite this article in press as: Frojan, P., et al. The continuous Berth Allocation Problem in a container terminal with multiple quays. *Expert Systems with Applications* (2015), http://dx.doi.org/10.1016/j.eswa.2015.05.018



(a) Vessel i is chosen to be moved (b) Vessels v_1 and v_2 are removed (c) Vessels v_1 and v_2 are reassigned

Fig. 6. Local search. Process of reassigning a high-cost vessel.

candidate from the set of candidates until a feasible assignment is found. The procedure is explained in more detail in Fig. 4. A partial solution composed of vessels v_1 and v_2 has already been built and the next vessel in the sequence, vessel i, has to be placed. In Fig. 4(a), vessel i is initially assigned to its best position, given by its arrival time and its desired position at the quay with the least assignment cost. As it overlaps both v_1 and v_2, another assignment must be found for vessel i. In Fig. 4(b), a vertical segment has been identified in which the bottom-left corner of vessel i cannot be placed. Using this segment and the position of v_1 and v_2, some new candidate positions for vessel i have been identified (in dashed lines). In Fig. 4(c), vessel i is assigned to the least-cost feasible position. In the general case, if all the candidate positions in Fig. 4(b) had been unfeasible, overlapping other vessels in the partial solution, the process would have continued, identifying new positions until a feasible position appeared.

5.2.2. Analytic Constructive Algorithm

The second algorithm, called the *Analytic Constructive Algorithm (ACA)*, develops an analysis of the preexisting plan for each quay. This analysis is based on the layout representation of the current partial assignment at each quay, which is built incrementally each time a vessel is assigned to the quay. The process can be seen in Fig. 5. Vessel i has to be placed, but when assigned to its best position (Fig. 5(a)) it overlaps vessels already assigned. Then, the analytic process starts. In a first phase, Fig. 5(b), the segments defining the layout representation are identified and their vertices are given a label indicating whether there is free space in any of the quadrants around them, as in Lee et al. (2010). The list of potential relevant points for assigning the next vessel includes these vertices and also some other points obtained by using the information about the vessel i being assigned. These points are identified by drawing vertical lines that extend the vertical segments of the representation, including a vertical line at a_i, and horizontal lines extending the horizontal segments, including a line at d_{iq}, as in Fig. 5(c). The intersections of these lines define new points, which are labeled in a similar way to the layout representation vertices. For each point identified in this process, the candidate assignments indicated by its label are obtained and put into a candidate set, ordered by their cost. The candidates are considered one at a time until a feasible assignment is found. In Fig. 5(d), vessel i is placed with its top-left corner on point A.

Comparing the two constructive algorithms, *ECA* is faster in the test instances we use, up to 200 vessels, though its computation time increases more rapidly with the number of vessels. This would make *ACA* more attractive if we had to deal with larger instances. Another aspect of the comparison is that *ECA* can be used if the constructive process starts from a partial solution, whereas *ACA* builds the layout representation in an incremental way and this representation would have to be built from scratch using the information from the partial solution. For these two reasons, in the genetic algorithm we have preferred to use the *Exploratory Constructive Algorithm*.

5.3. Improving the best solution obtained through a local search procedure

The genetic algorithm works on ordered lists of vessels. As a last step in the algorithm we have designed a local search phase to explore solutions that are neighbors of the best solution known but do not correspond to orderings of the list of vessels. The move defining the neighborhood of the solution consists of choosing at random a vessel i with positive cost, that is, a vessel that is not assigned to its best quay, time, and position, and moving it to the position of another vessel k if vessel i fits into this position and its assignment cost is decreased. The new position of vessel i will produce an overlap with vessel k and possibly with other vessels. All these vessels are removed from the solution and assigned again by the deterministic constructive procedure. In order to reduce the search, not all the vessels are considered, but only a given percentage, $h\%$ of the vessels with the highest costs. These vessels are taken one at a time and the corresponding move is studied. If one of them produces an improved solution, the best solution known is updated and the search starts again from that solution. If all the selected vessels are studied and no improvement is found, the search stops. Fig. 6 shows the procedure in more detail. Note that in Fig. 6, if the desired position of vessels v_1 and i was the bottom left corner of the position of vessel v_1 in the initial solution and the desired position of vessels v_2 and v_3 was the bottom left corner of the position of vessel v_2, the solution produced after the move could not be obtained with any ordering of vessels.

6. Computational study

In order to study the performance of the algorithms developed in previous sections, we have designed an extensive computational study. First, we study the behavior and the limits of using the proposed integer model for solving the problem exactly. We use the instances by Park and Kim (2002), modified to accommodate several quays, as well as the instances by Lee et al. (2010) and by Cordeau et al. (2005). Then, we analyze and compare the results obtained by the genetic algorithm and the constructive algorithm *ECA* with the 26 priority rules. We also compare our genetic algorithm with those developed by Lee et al. (2010) and by Cordeau et al. (2005). Finally, we describe a test instance generator we have built to generate instances with controlled characteristics. Studying the results obtained by the algorithms on a set of newly generated instances allows us to get some insight into the more

UNCORRECTED PROOF

Please cite this article in press as: Frojan, P., et al. The continuous Berth Allocation Problem in a container terminal with multiple quays. *Expert Systems with Applications* (2015), http://dx.doi.org/10.1016/j.eswa.2015.05.018

Table 2
Solving the instances of Park and Kim (2002) to optimality.

	Class	Average	Median	Maximum
1 Quay	Real-world	0.08	0.08	0.09
	20	0.14	0.09	0.27
	25	0.16	0.14	0.28
	30	0.21	0.16	0.37
	35	1.10	0.66	3.23
	40	536.55	3.50	5148.36
2 Quays	Real-world	0.12	0.10	0.23
	20	0.17	0.15	0.37
	25	0.49	0.30	1.69
	30	0.74	0.32	2.87
	35	4.53	3.99	12.18
	40	28.47	13.38	118.75
3 Quays	Real-world	0.12	0.10	0.22
	20	0.14	0.13	0.20
	25	0.25	0.26	0.37
	30	0.29	0.24	0.58
	35	1.27	0.88	2.93
	40	13.74	4.88	68.22

influential factors associated with the problem. The algorithms were coded in C++ (Visual Studio 2012) and run on a PC with an Intel (R) Core(TM) i7-2600 3.40 GHz, with 12 GB of RAM.

6.1. Results of the integer linear model

Park and Kim (2002) generated a set of test instances taking the Pusan Eastern Container Terminal in Pusan, South Korea, as a reference. The quay is 1200 meters long. First they generated 25 instances ranging from 13 to 20 vessels, referred to as real-world instances, and then 50 instances, 10 for each size: 20, 25, 30, 35, and 40 vessels. Estimated arrival times, handling times, vessel lengths, and costs are drawn from uniform distributions in an attempt to capture the values of real-world problems. We have modified the original data in order to extend the model to the case of multiple quays. So, besides the 75 instances with one quay, we have generated two more sets with two and three quays, with the same vessels, splitting the length of the original quay into two or three parts at random, but ensuring that the shortest quay is able to accommodate at least the shortest vessel.

The results are shown in Table 2 and were obtained using *CPLEX 12.6* on an Intel Quad Core at 2.5 GHz. The table has three parts, corresponding to the number of quays. The instances have been divided into classes. The first corresponds to the instances labeled by Park and Kim (2002) as real-world instances, with the number of vessels between 13 and 20. The other classes correspond to the generated instances with 20–40 vessels. For each class, the table shows the average, median, and maximum time required to solve the instances to optimality. It can be seen on the last line of the first section that there is one instance with 40 vessels which is very hard to solve. Apart from this exceptional case, the number of quays does not seem to have a significant effect on the computing times. On the contrary, the required times increase very sharply with the number of vessels, but instances of up to 40 vessels can be solved in reasonable times even in the worst cases.

In order to extend the study of the integer model, we also considered the instances generated by Lee et al. (2010). They generated large instances with 40, 80, 120, 160, and 200 vessels for the problem with just one quay. We started by solving the 40 vessel instances, with a time limit of 3600 s per instance, and none of them was solved to optimality within this time limit. Moreover, the average gap was 78%. These instances are so hard for the integer linear code because the arrival times of all the vessels are concentrated at the beginning of the planning period, producing a congestion which appears to be very difficult to manage. This congestion factor is analyzed in more detail in Section 6.5.

Table 3
Results of the genetic algorithm.

		Algorithm		
		GA	GA + LS	Best rule
1 Quay	Average deviation	0.08	0.08	3.29
	Optimal solutions	72	72	62
2 Quays	Average deviation	1.14	1.14	12.73
	Optimal solutions	69	69	56
3 Quays	Average deviation	0.22	0.22	15.08
	Optimal solutions	72	72	47

We also solved the instances generated by Cordeau et al. (2005), with one quay and 60 vessels, again with a time limit of 3600 s per instance. Only one of the 30 instances was solved to optimality within the time limit. The average gap was 6.6%. Compared with the Lee et al. (2010) instances, these seem to be easier, but they are larger, and 60 vessels appear to be too many for the integer code. In these two sets of instances, solving the integer model does not provide good solutions in reasonable computing times and therefore an efficient metaheuristic algorithm is needed for solving large problems.

6.2. Results of the genetic algorithm

Table 3 compares the results obtained on the Park and Kim instances by the variants of the genetic algorithm: genetic algorithm alone (GA) and genetic algorithm plus a Local Search phase applied on the best solution provided by the GA (GA + LS), with the best result obtained when applying the 26 priority rules (Best Rule). The rows show the average percentage deviation from the optimal solution and the number of optimal solutions for the 75 instances with one, two, and three quays. The genetic algorithm stops after having completed 100 consecutive generations without improving on the best solution known.

Table 3 shows that the genetic algorithm performs very well on these instances, obtaining the optimal solution in all but 12 of the 225 test instances. Looking at the results in Table 3 we could conclude that we do not need to add the local search phase to the genetic algorithm, because it does not produce any improvement. However, as the local search does not increase the computing times very much, we have kept it in the final version of our metaheuristic algorithm, because it could be useful when solving larger and more difficult instances, tested in the next sections.

The last column of the table shows the results when we solve each instance with the 26 priority rules and keep the best solution. The number of optimal solutions is relatively high, though it decreases with the number of quays, while the average distance to the optimal solution is high for one-quay instances and increases very sharply with the number of quays. That indicates that this set contains many instances that are easy to solve by any algorithm, but also hard instances for which only complex metaheuristics, like our genetic algorithm, are able to obtain optimal or near-optimal solutions, while simple rules or a combination of them fail by a large margin.

Table 4
Small instances of Lee et al.

Vessels	*CPLEX*	GRASP1		GRASP2		GA	
	T	GAP	*T*	GAP	*T*	GAP	*T*
5	0.39	1.62	2.29	0.005	2.73	0.15	0.04
10	5977	7.89	11.90	3.70	16.59	0.72	0.09

UNCORRECTED PROOF

Please cite this article in press as: Frojan, P., et al. The continuous Berth Allocation Problem in a container terminal with multiple quays. *Expert Systems with Applications* (2015), http://dx.doi.org/10.1016/j.eswa.2015.05.018

Table 5
Large instances by Lee et al.

Vessels	SBS		GRASP1		GRASP2		GA (t.l.)		GA	
	Value	*T*	Value	*T*	Value	*T*	Value	*T*	Value	*T*
40	15,094	188	14,958	15	14,766	37	12,063	1.6	11,875	5
80	60,132	790	58,858	112	56,564	271	46,231	11	45,135	59
120	134,300	911	129,852	413	125,023	882	103,477	38	101,108	187
160	237,358	1703	229,432	1103	218,520	2137	181,873	92	178,674	411
200	374,808	2809	364,986	1763	344,793	4223	288,312	184	281,849	668

6.3. The Lee et al. instances

Lee et al. (2010) consider the same problem we are studying in this paper, but with only one quay and with the single objective of minimizing the total servicing time of the vessels, defined as the elapsed time from their arrival to the end of their handling. They develop an integer linear model and two GRASP algorithms.

In a first computational study on 30 small problems with 5 and 10 vessels, they compare the results obtained by solving the integer linear model with *CPLEX* and using GRASP1 and GRASP2. The results are shown in Table 4, taken from Lee et al. (2010), to which we have added the last column with the results of our genetic algorithm, adapted to consider only one quay and the objective defined by them. For each algorithm, the table shows the execution time (*T*) and the average percentage deviation from the optimum (GAP). It can be seen that solving their integer model is very costly, even for problems with 10 vessels. For these small instances our genetic algorithm obtains solutions very close to optimal, not as close as those of GRASP2 for 5-vessel instances, but clearly better for 10-vessel problems, in very short computing times.

In a second study, Lee et al. (2010) use larger instances with 40, 80, 120, 160, and 200 vessels, 30 instances of each size. Table 5, also built by using the data kindly provided by the authors, uses the results of the previously published *SBS* (Stochastic Beam Search) algorithm by Wang as an initial reference, and shows the results of the *GRASP1* and *GRASP2* algorithms proposed by Lee et al. (2010). The right-hand side of the table shows the results obtained by two versions of our algorithm. First, we ran the genetic algorithm (including local search) with a fixed time limit, giving our algorithm a running time as similar as possible to that used by *GRASP2*. As our computer, according to http://www.cpubenchmark.net, is approximately 23 times faster, the time limits have been set to the running times reported by Lee et al. (2010) divided by 23. Finally, the last two columns show the results and time of our algorithm with the stopping criterion described in the previous section. If we consider the controlled-time version of *GA*, the results are 17.4% better on average than *GRASP2*. The *GA*, with the stopping criterion of 100 generations without improvement, requires more time and gets solutions which are on average 19.1% lower than those obtained by *GRASP2*.

6.4. The Cordeau et al. instances

Cordeau et al. (2005) study the discrete problem first, considering only costs related to the total time the vessels are moored at the dock, from arrival to the end of their servicing. They propose an integer linear model and a Tabu Search algorithm, which is later adapted to the continuous case. For this continuous case they generate 30 instances with only 1 quay and 60 vessels, taking the real problems encountered at the Gioia Tauro terminal in Italy as a reference. For these instances, the authors present the results of their Tabu Search algorithm and compare them with a FSFC-G algorithm which orders the vessels by non-decreasing arrival time and assigns each vessel to the position and mooring time that minimize the time needed for it to be serviced. According to the authors, FCFS-G is similar to the procedure actually used at the terminal.

Table 6
Instances by Cordeau et al.

Inst.	FCFS-G	TS	GA	Inst.	FCFS-G	TS	GA
1	1899	1706	1531	16	1854	1715	1406
2	1417	1355	1277	17	1388	1322	1293
3	1349	1286	1152	18	1923	1594	1395
4	1548	1440	1333	19	1829	1673	1425
5	1449	1352	1227	20	1615	1450	1351
6	1747	1565	1251	21	1640	1565	1377
7	1482	1389	1290	22	1747	1618	1355
8	1616	1519	1366	23	1770	1539	1329
9	1873	1713	1516	24	1625	1425	1297
10	1611	1411	1206	25	1845	1590	1454
11	1851	1696	1417	26	1707	1567	1388
12	1814	1629	1414	27	1588	1458	1273
13	1575	1519	1384	28	1669	1550	1414
14	1435	1369	1240	29	1512	1415	1286
15	1609	1455	1315	30	1797	1621	1446

Table 6 shows the results of the FCFS-G and Tabu Search algorithm by Cordeau et al. (2005), as well as the results obtained by our genetic algorithm. For all the instances, our GA improves on the results of both FCFS-G and TS, achieving an average improvement of 18% over FCFS-G and 11% over TS. The average running time is 2.2 s.

6.5. A generator of random instances

All the instances for the BAP published in the literature consider just one quay. In order to study the problem with multiple quays, we have developed an instance generator which accepts all the relevant characteristics of the problem as configurable parameters. We have used it to study the effect of the key factors on the complexity of the problem and the performance of the algorithms.

In this study, some parameters have been fixed:

- Maximum length of a quay (L_{max}): 1500.
- Minimum length of a quay (L_{min}): 100.
- Maximum waiting cost per time unit (C^w): 5.
- Maximum delay cost per time unit (C^d): 10.
- Maximum assignment cost of vessel to quay (C^a): 10.
- Maximum cost of deviation from ideal position at quay per unit (C^p): 3.
- Slack for departure time ($s - a - b$): 5.

For each instance the length of each quay is then randomly drawn from (100, 1500) and the costs for each vessel are randomly taken in the interval between 0 and the maximum cost.

For the other parameters, we consider those that most influence the complexity of the problem and therefore the behavior of the algorithms, and we include several levels:

UNCORRECTED PROOF

Please cite this article in press as: Frojan, P., et al. The continuous Berth Allocation Problem in a container terminal with multiple quays. *Expert Systems with Applications* (2015), http://dx.doi.org/10.1016/j.eswa.2015.05.018

- Number of quays: 1, 3, 5.
- Number of vessels: 20, 30, 40.
- Length of vessels (Minimum, Maximum): (10, 100), (10, 250), (10, 500).
- Handling times (Minimum, Maximum): (1, 10), (1, 25), (25, 50).
- Maximum arrival time: 20, 50, 100.

Using the generator we have generated 1215 instances, 5 replications for each combination of the parameter levels. In this way, besides the effect of factors such as the number of quays and vessels, we can study different configurations involving the type of vessels (all small, of varying sizes), the handling times (all short, of different lengths, all large), and the arrival time (all at the beginning of the planning interval or more scattered over time).

The first part of our study focuses on the effect of these factors on the ability of our integer model to obtain the optimal solution for each type of problem. Each one of the 1215 instances was solved by using *CPLEX* 12.3 with a time limit of 600 s. Table 7 shows the number of optimal solutions for each number of quays and each number of vessels. We can observe first that these instances are much harder to solve than the instances of Park and Kim (2002), in which all but one of the instances were solved to optimality in less than 600 s. For the instances generated here, only 731 are solved within the time limit (60%). We can also see that, as expected, the number of vessels has a strong influence on the results. For 20, 30, and 40 vessels, the percentage of problems solved was 80%, 58%, and 43% respectively. Also, and more surprisingly, the number of quays has an effect: the more quays there are, the easier it is to solve (39%, 63%, 79% for 1, 3, 5 quays respectively). This effect was not observed in previous tests. The main difference may lie in the fact that here we have added assignment costs of vessels to quays and deviation costs from the ideal position at the quays. That seems to introduce some structure into the problem which is used by *CPLEX* to obtain the optimal solution faster when several quays are involved.

Table 8 shows the effect of the length and the handling times of the vessels. Both factors have a clear influence on the results. Problems with large vessels and large handling times are very hard to solve to optimality.

Finally, Table 9 shows the effect of the distribution of the arrival times, which could be called the congestion factor. If all the vessels arrive at the same time, the problem is much harder than if the arrivals are scattered over the time horizon.

In the second part of the study, we solved each of the 1215 instances by using the genetic algorithm with a stopping criterion of 100 CPU seconds. For each instance, we calculated the percentage distance to the solution provided by *CPLEX*. In Table 10 we show the average results classified by quays and vessels, separating into two columns the cases in which the optimal solution was found from those in which *CPLEX* only returned a feasible solution.

Looking at the column of the instances for which *CPLEX* obtained the optimal solution, the solutions obtained by the genetic algorithm are on average 1.2% higher, getting nearer as the number of quays increases. For the instance in which *CPLEX* did not obtain the optimal solution within the time limit, the average results clearly favor the genetic algorithm, which obtains much better results in instances with larger numbers of vessels. For the 484 instances in which *CPLEX* did not get the optimal solution in 600 s, the genetic algorithm obtained a solution better than *CPLEX* in 340: 179 for 1-quay instances, 100 for 3-quay instances, and 61 for 5-quay instances.

Table 7
Optimal solutions by quays and vessels.

Quays	Vessels			Total
	20	30	40	
1	84	45	27	**156**
3	112	83	62	**257**
5	126	105	87	**318**
Total	**322**	**233**	**176**	**731**

Table 8
Optimal solutions by lengths and handling times of the vessels.

Handling time	Length			Total
	(1, 10)	(1, 25)	(1, 50)	
(1, 10)	128	114	85	**327**
(1, 25)	118	85	59	**262**
(25, 50)	92	40	10	**142**
Total	**338**	**239**	**154**	**731**

Table 9
Optimal solutions by quays, vessels, and arrival times.

Quays	Vessels	Arrival time			Total
		(1, 20)	(1, 50)	(1, 100)	
1		**26**	**55**	**75**	**156**
	20	17	32	35	**84**
	30	6	15	24	**45**
	40	3	8	16	**27**
3		**60**	**87**	**110**	**257**
	20	35	36	41	**112**
	30	14	32	37	**83**
	40	11	19	32	**62**
5		**89**	**105**	**124**	**318**
	20	40	41	45	**126**
	30	28	36	41	**105**
	40	21	28	38	**87**
Total		**175**	**247**	**309**	**731**

Table 10
Average percentage distance from *GA* to *CPLEX*.

Quays	Vessels	Optimal solutions	Non-optimal solutions	Total
1		**3.1**	**−7.0**	**−3.1**
	20	3.8	3.6	3.8
	30	2.8	−6.8	−3.6
	40	1.4	−12.2	−9.5
3		**0.9**	**−6.3**	**−1.8**
	20	1.1	4.9	1.8
	30	0.8	−6.7	−2.1
	40	0.5	−9.6	−5.0
5		**0.4**	**−6.6**	**−1.1**
	20	0.8	8.6	1.3
	30	0.2	−7.1	−1.4
	40	0.3	−9.1	−3.0
Total		**1.2**	**−6.7**	**−2.0**

7. Conclusions

The Berth Allocation Problem is an important part of the container terminal operational system and it has been extensively studied, in its discrete and continuous versions. Having a terminal divided into several separated quays is not a problem for the discrete version, because the terminal, and therefore each of its quays, is divided into sections which can accommodate at most one vessel. However, a terminal with several quays is a problem for the continuous version, because in almost all the studies the terminal is considered as one straight line in which vessels can be berthed

UNCORRECTED PROOF

Please cite this article in press as: Frojan, P., et al. The continuous Berth Allocation Problem in a container terminal with multiple quays. *Expert Systems with Applications* (2015), http://dx.doi.org/10.1016/j.eswa.2015.05.018

according to their length and the positions of other vessels. In this study we have considered for the first time the continuous Berth Allocation problem with several quays at the operational level. Besides the usual costs of waiting to be berthed and of delay with respect to the ideal departure date, we have included an assignment cost of vessel to quay (which can be used to forbid some vessel–quay combinations) and a deviation cost with respect to the ideal position of the vessel at the quay to which it is actually assigned.

We have developed first an integer linear model for the problem, extending previous models for the one-quay case. The model can be used to obtain the optimal solution in reasonable computing times for some instances with up to 40 vessels (Park & Kim, 2002), but we have identified several classes of instances for which obtaining the optimal solution is very hard. We have therefore developed a genetic algorithm, based on sequences of vessels. The initial population is built by applying a large set of priority rules which produce good and diverse solutions. For decoding vessel sequences into BAP solutions, we have developed two fast and efficient constructive procedures. The solutions produced by the genetic algorithm are improved by a local search procedure.

As there are no similar studies for multiple quays, in order to assess the performance of our procedure, we have applied it to three sets of benchmark instances for the case of one quay, and the computational study shows that our algorithm outperforms the best results recently published in the literature for the one-quay case.

The problem studied in this paper is limited to the description and assumptions set out in Section 3. Here we have dealt with the Berth Allocation Problem, which considers fixed handling times, without considering how it is related to the Quay Crane Assignment Problem and the Quay Crane Scheduling Problem, which are also part of the seaside terminal operating system.

There are several ways in which this study could be extended. In a first extension of this work, the model and algorithms for multiple quays developed here will be extended to the Berth Allocation and Crane Assignment Problem (BACAP), which combines the allocation of berths to vessels with the allocation of quay cranes to serve those vessels. A further step would be to connect it with the Quay Crane Scheduling Problem. Other possible extensions of the study would involve considering it as a multi-objective problem. Instead of combining all the costs into a single objective function, conflicting objectives can be studied separately, using a multi-objective procedure, providing users with a set of non-dominated solutions. The problem can also be considered at a more tactical or strategic level, if instead of considering a list of actual vessels, with their specific characteristics, we consider at a higher level the problem of shipping lines with sets of vessels cyclically calling at the port that have to be assigned to several quays or even to several terminals, taking into account all the operational and transportation costs involved.

Acknowledgments

The authors would like to thank Young-Man Park, Der-Horng Lee, and Pasquale Legato for sending us their instances and results. They would also like to express their appreciation to the reviewers for making valuable comments and suggestions to improve the paper.

This study has been partially supported by the Spanish Ministry of Economy and Competitiveness, DPI2011-24977, and by Generalitat Valenciana, PROMETEO/2013/049, Grant CPI-13-351.

References

Bierwirth, C., & Meisel, F. (2010). A survey of berth allocation and quay crane scheduling problem in container terminals. *European Journal of Operational Research, 202*, 615–627.

Bierwirth, C., & Meisel, F. (2015). A follow-up survey of berth allocation and quay crane scheduling problem in container terminals. *European Journal of Operational Research, 244*, 675–689.

Cheong, C., Tan, K., Liu, D., & Lin, C. (2010). Multi-objective and prioritized berth allocation in container ports. *Annals of Operations Research, 180*, 63–103.

Cordeau, J., Laporte, G., Legato, P., & Moccia, L. (2005). Models and tabu search heuristics for the berth allocation problem. *Transportation Science, 39*, 526–538.

Dai, J., Lin, W., Moorthy, R., & Teo, C. (2008). Berth allocation planning optimization in container terminals. In C. Tang, C. Teo, & K. Wei (Eds.), *Supply chain analysis: A handbook on the interaction of information, system and optimization*. Springer.

Du, Y., Chen, Q., Quan, X., Long, L., & Fung, R. (2011). Berth allocation considering fuel consumption and vessel emissions. *Transportation Research Part E: Logistics and Transportation Review, 47*, 1021–1037.

Gonçalves, J., & Resende, M. (2013). A biased random key genetic algorithm for 2D and 3D bin packing problems. *International Journal of Production Economics, 145*, 500–510.

Guan, Y., & Cheung, R. (2004). The berth allocation problem: Models and solution methods. *OR Spectrum, 26*, 75–92.

Guan, Y., Xiao, W., Cheung, R., & Li, C. (2002). A multiprocessor task scheduling model for berth allocation: Heuristic and worst-case analysis. *Operations Research Letters, 30*, 343–350.

Hendriks, M., Laumanns, M., Lefeber, E., & Udding, J. (2010). Strategic allocation of cyclically calling vessels for multi-terminal container operations. *Flexible Services and Manufacturing Journal, 24*, 248–273.

Hopper, E., & Turton, B. (2001). An empirical investigation of meta-heuristic and heuristic algorithms for a 2D packing problem. *European Journal of Operational Research, 128*, 34–57.

Imai, A., Nishimura, E., & Papadimitriou, S. (2013). Marine container terminal configurations for efficient handling of mega-containerships. *Transportation Research Part E: Logistics and Transportation Review, 49*, 141–158.

Iori, M., Martello, S., & Monaci, M. (2003). Metaheuristic algorithms for the strip packing problem. In P. Pardalos & V. Korotkikh (Eds.), *Optimization in industry: New frontiers*. Kluwer Academic Publishers.

Kim, K., & Moon, K. (2003). Berth scheduling by simulated annealing. *Transportation Research. Part B, Methodological, 37*, 541–560.

Lee, D.-H., Chen, J., & Cao, J. (2010). The continuous berth allocation problem: A greedy randomized adaptive search solution. *Transportation Science, 46*, 1017–1029.

Lee, D.-H., Jin, J., & Chen, J. (2012). Terminal and yard allocation problem for a container transshipment hub with multiple terminals. *Transportation Research Part E: Logistics and Transportation Review, 48*, 516–528.

Li, C., Cai, X., & Lee, C. (1998). Scheduling with multiple-job-on-one-processor pattern. *IIE Transactions, 30*, 433–445.

Lim, A. (1998). The berth planning problem. *Operations Research Letters, 22*, 105–110.

Nishimura, E., Imai, A., & Papadimitriou, S. (2001). Berth allocation planning in the public berth system by genetic algorithms. *European Journal of Operational Research, 131*, 282–292.

Oliveira, R. de, Mauri, G., & Lorena, L. (2012). Clustering search for the berth allocation problem. *Expert Systems with Applications, 39*, 5499–5505.

Park, K., & Kim, K. (2002). Berth scheduling for container terminals by using a subgradient optimization technique. *Journal of the Operational Research Society, 53*, 1054–1062.

Ting, C., Wu, K., & Chou, H. (2014). Particle swarm optimization algorithm for the berth allocation problem. *Expert Systems with Applications, 41*, 1543–1550.

Tong, C., Lau, H., & Lim, A. (1999). Ant colony optimization for the ship berthing problem. *Lecture notes in computer science* (Vol. 1742, pp. 359–370). Springer.

UNCTAD. (2014). Review of maritime transport 2014, New York, United Nations Publication.

Vallada, E., & Ruiz, R. (2011). A genetic algorithm for the unrelated parallel machine scheduling problem with sequence dependent setup times. *European Journal of Operational Research, 211*, 612–622.

Wang, F., & Lim, A. (2007). A stochastic beam search for the berth allocation problem. *Decision Support Systems, 42*, 2186–2196.

Xu, Y., Chen, Q., & Quan, X. (2012). Robust berth scheduling under uncertain vessel delay and handling time. *Annals of Operations Research, 192*, 123–140.

Zhen, L., & Chang, D. (2012). A bi-objective model for robust berth allocation scheduling. *Computers & Industrial Engineering, 63*, 262–273.

UNCORRECTED PROOF